

FinalProject__AUSWeather

September 17, 2026

=

1 Final Project: Building a Rainfall Prediction Classifier

Estimated time needed: **60** minutes

1.1 Objectives

After completing this lab you will be able to:

- Explore and perform feature engineering on a real-world data set
- Build a classifier pipeline and optimize it using grid search cross validation
- Evaluate your model by interpreting various performance metrics and visualizations
- Implement a different classifier by updating your pipeline
- Use an appropriate set of parameters to search over in each case

1.2 Instruction(s)

After completing the Notebook:

- Download the notebook using **File > Download**.
- This notebook will be then graded using **AI grader** in the subsequent section.
- Copy/Paste your markdown responses in the subsequent **AI Mark assignment**.

2 About The Dataset

The original source of the data is Australian Government's Bureau of Meteorology and the latest data can be gathered from <http://www.bom.gov.au/climate/dwo/>.

The dataset you'll use in this project was downloaded from Kaggle at <https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package/>

Column definitions were gathered from <http://www.bom.gov.au/climate/dwo/IDCJDW0000.shtml>

The dataset contains observations of weather metrics for each day from 2008 to 2017, and includes the following fields:

Field	Description	Unit	Type
Date	Date of the Observation in YYYY-MM-DD	Date	object
Location	Location of the Observation	Location	object
MinTemp	Minimum temperature	Celsius	float

Field	Description	Unit	Type
MaxTemp	Maximum temperature	Celsius	float
Rainfall	Amount of rainfall	Millimeters	float
Evaporation	Amount of evaporation	Millimeters	float
Sunshine	Amount of bright sunshine	hours	float
WindGustDir	Direction of the strongest gust	Compass Points	object
WindGustSpeed	Speed of the strongest gust	Kilometers/Hour	object
WindDir9am	Wind direction averaged over 10 minutes prior to 9am	Compass Points	object
WindDir3pm	Wind direction averaged over 10 minutes prior to 3pm	Compass Points	object
WindSpeed9am	Wind speed averaged over 10 minutes prior to 9am	Kilometers/Hour	float
WindSpeed3pm	Wind speed averaged over 10 minutes prior to 3pm	Kilometers/Hour	float
Humidity9am	Humidity at 9am	Percent	float
Humidity3pm	Humidity at 3pm	Percent	float
Pressure9am	Atmospheric pressure reduced to mean sea level at 9am	Hectopascal	float
Pressure3pm	Atmospheric pressure reduced to mean sea level at 3pm	Hectopascal	float
Cloud9am	Fraction of the sky obscured by cloud at 9am	Eights	float
Cloud3pm	Fraction of the sky obscured by cloud at 3pm	Eights	float
Temp9am	Temperature at 9am	Celsius	float
Temp3pm	Temperature at 3pm	Celsius	float
RainToday	If there was at least 1mm of rain today	Yes/No	object
RainTomorrow	If there is at least 1mm of rain tomorrow	Yes/No	object

2.1 Install and import the required libraries

Execute the following cells to install and import the necessary libraries.

```
[63]: !pip install numpy
      !pip install pandas
      !pip install matplotlib
      !pip install scikit-learn
      !pip install seaborn
```

```
Requirement already satisfied: numpy in /opt/conda/lib/python3.12/site-packages
(2.5.3) Requirement already satisfied: pandas in /opt/conda/lib/python3.12/site-
packages (3.0.5)
Requirement already satisfied: numpy>=1.26.0 in /opt/conda/lib/python3.12/site-
packages (from pandas) (2.5.3)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/conda/lib/python3.12/site-packages (from pandas) (2.9.0.post0) Requirement
already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0) Requirement already satisfied:
matplotlib in /opt/conda/lib/python3.12/site-packages (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in
/opt/conda/lib/python3.12/site-packages (from matplotlib) (1.4.0)
```

Requirement already satisfied: cycloper>=0.10 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.28.2 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (4.65.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (1.5.1)

Requirement already satisfied: numpy>=1.25 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (2.5.3)

Requirement already satisfied: packaging>=20.0 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (25.0)

Requirement already satisfied: pillow>=9 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (12.3.0)

Requirement already satisfied: pyparsing>=3 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.12/site-packages (1.9.1)

Requirement already satisfied: numpy>=1.24.1 in /opt/conda/lib/python3.12/site-packages (from scikit-learn) (2.5.3)

Requirement already satisfied: scipy>=1.10.0 in /opt/conda/lib/python3.12/site-packages (from scikit-learn) (1.18.1)

Requirement already satisfied: joblib>=1.4.0 in /opt/conda/lib/python3.12/site-packages (from scikit-learn) (1.6.0)

Requirement already satisfied: narwhals>=2.0.1 in /opt/conda/lib/python3.12/site-packages (from scikit-learn) (2.26.0)

Requirement already satisfied: threadpoolctl>=3.5.0 in /opt/conda/lib/python3.12/site-packages (from scikit-learn) (3.7.0)

Requirement already satisfied: cloudpickle>=3.0 in /opt/conda/lib/python3.12/site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)

Requirement already satisfied: seaborn in /opt/conda/lib/python3.12/site-packages (0.13.2)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/conda/lib/python3.12/site-packages (from seaborn) (2.5.3)

Requirement already satisfied: pandas>=1.2 in /opt/conda/lib/python3.12/site-packages (from seaborn) (3.0.5)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/conda/lib/python3.12/site-packages (from seaborn) (3.11.2)

Requirement already satisfied: contourpy>=1.0.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.0)

Requirement already satisfied: cycloper>=0.10 in /opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: fonttools>=4.28.2 in /opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.65.0)

Requirement already satisfied: kiwisolver>=1.3.1 in

```

/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(1.5.1)
Requirement already satisfied: packaging>=20.0 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(25.0)
Requirement already satisfied: pillow>=9 in /opt/conda/lib/python3.12/site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in /opt/conda/lib/python3.12/site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(2.9.0.post0) Requirement already satisfied: six>=1.5 in
/opt/conda/lib/python3.12/site-packages (from python-
dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

```

```

[64]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split, GridSearchCV,
↳StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,
↳ConfusionMatrixDisplay
import seaborn as sns

```

2.2 Load the data

Execute the following cells to load the dataset as a pandas dataframe.

```

[65]: url="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
↳_0eY0qji3unP1tDNKWZMjg/weatherAUS-2.csv"
df = pd.read_csv(url)
df.head()

```

```

[65]:
      Date Location  MinTemp  MaxTemp  Rainfall  Evaporation  Sunshine  \
0  2008-12-01  Albury    13.4    22.9        0.6           NaN         NaN
1  2008-12-02  Albury     7.4    25.1        0.0           NaN         NaN
2  2008-12-03  Albury    12.9    25.7        0.0           NaN         NaN
3  2008-12-04  Albury     9.2    28.0        0.0           NaN         NaN
4  2008-12-05  Albury    17.5    32.3        1.0           NaN         NaN

      WindGustDir  WindGustSpeed  WindDir9am  ...  Humidity9am  Humidity3pm  \
0              W             44.0           W  ...        71.0         22.0
1            WNW             44.0          NNW  ...        44.0         25.0
2            WSW             46.0           W  ...        38.0         30.0

```

3	NE	24.0	SE ...	45.0	16.0
4	W	41.0	ENE ...	82.0	33.0

	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	Temp9am	Temp3pm	RainToday \
0	1007.7	1007.1	8.0	NaN	16.9	21.8	No
1	1010.6	1007.8	NaN	NaN	17.2	24.3	No
2	1007.6	1008.7	NaN	2.0	21.0	23.2	No
3	1017.6	1012.8	NaN	NaN	18.1	26.5	No
4	1010.8	1006.0	7.0	8.0	17.8	29.7	No

	RainTomorrow
0	No
1	No
2	No
3	No
4	No

[5 rows x 23 columns]

```
[66]: df.count()
```

```
[66]: Date          145460
      Location      145460
      MinTemp       143975
      MaxTemp       144199
      Rainfall      142199
      Evaporation    82670
      Sunshine      75625
      WindGustDir    135134
      WindGustSpeed  135197
      WindDir9am     134894
      WindDir3pm     141232
      WindSpeed9am   143693
      WindSpeed3pm   142398
      Humidity9am    142806
      Humidity3pm    140953
      Pressure9am    130395
      Pressure3pm    130432
      Cloud9am       89572
      Cloud3pm       86102
      Temp9am        143693
      Temp3pm        141851
      RainToday      142199
      RainTomorrow   142193
      dtype: int64
```

Sunshine and cloud cover seem like important features, but they have a lot of missing values, far too many to impute their missing values.

2.2.1 Drop all rows with missing values

To try to keep things simple we'll drop rows with missing values and see what's left

```
[67]: df = df.dropna()
      df.info()

<class 'pandas.DataFrame'>
Index: 56420 entries, 6049 to 142302
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                   56420 non-null  str
1   Location               56420 non-null  str
2   MinTemp                56420 non-null  float64
3   MaxTemp                56420 non-null  float64
4   Rainfall               56420 non-null  float64
5   Evaporation            56420 non-null  float64
6   Sunshine               56420 non-null  float64
7   WindGustDir            56420 non-null  str
8   WindGustSpeed          56420 non-null  float64
9   WindDir9am             56420 non-null  str
10  WindDir3pm             56420 non-null  str
11  WindSpeed9am           56420 non-null  float64
12  WindSpeed3pm           56420 non-null  float64
13  Humidity9am            56420 non-null  float64
14  Humidity3pm            56420 non-null  float64
15  Pressure9am            56420 non-null  float64
16  Pressure3pm            56420 non-null  float64
17  Cloud9am               56420 non-null  float64
18  Cloud3pm               56420 non-null  float64
19  Temp9am                56420 non-null  float64
20  Temp3pm                56420 non-null  float64
21  RainToday              56420 non-null  str
22  RainTomorrow           56420 non-null  str
dtypes: float64(16), str(7)
memory usage: 12.0 MB
```

Since we still have 56k observations left after dropping missing values, we may not need to impute any missing values.

Let's see how we do.

```
[68]: df.columns

[68]: Index(['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation',
          'Sunshine', 'WindGustDir', 'WindGustSpeed', 'WindDir9am', 'WindDir3pm',
          'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',
          'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm', 'Temp9am',
          'Temp3pm', 'RainToday', 'RainTomorrow'],
```

```
dtype='str')
```

2.3 Data leakage considerations

Consider the descriptions above for the columns in the data set. Are there any practical limitations to being able to predict whether it will rain tomorrow given the available data?

2.4 Points to note - 1

List some of the features that would be inefficient in predicting tomorrow's rainfall. There will be a question in the quiz that follows based on this observation.

[Click here for Hint](#)

Consider features that rely on the entire duration of today for their evaluation.

If we adjust our approach and aim to predict today's rainfall using historical weather data up to and including yesterday, then we can legitimately utilize all of the available features. This shift would be particularly useful for practical applications, such as deciding whether you will bike to work today.

With this new target, we should update the names of the rain columns accordingly to avoid confusion.

```
[69]: df = df.rename(columns={'RainToday': 'RainYesterday',  
                             'RainTomorrow': 'RainToday'  
                           })
```

2.5 Data Granularity

Would the weather patterns have the same predictability in vastly different locations in Australia? I would think not.

The chance of rain in one location can be much higher than in another. Using all of the locations requires a more complex model as it needs to adapt to local weather patterns.

Let's see how many observations we have for each location, and see if we can reduce our attention to a smaller region.

2.6 Location selection

You could do some research to group cities in the `Location` column by distance, which I've done for you behind the scenes.

I found that Watsonia is only 15 km from Melbourne, and the Melbourne Airport is only 18 km from Melbourne.

Let's group these three locations together and use only their weather data to build our localized prediction model.

Because there might still be some slight variations in the weather patterns we'll keep `Location` as a categorical variable.

```
[70]: df = df[df.Location.isin(['Melbourne', 'MelbourneAirport', 'Watsonia',])]  
df.info()
```

```

<class 'pandas.DataFrame'>
Index: 7557 entries, 64191 to 80997
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                   7557 non-null   str
1   Location                7557 non-null   str
2   MinTemp                 7557 non-null   float64
3   MaxTemp                 7557 non-null   float64
4   Rainfall                7557 non-null   float64
5   Evaporation             7557 non-null   float64
6   Sunshine                7557 non-null   float64
7   WindGustDir             7557 non-null   str
8   WindGustSpeed           7557 non-null   float64
9   WindDir9am              7557 non-null   str
10  WindDir3pm              7557 non-null   str
11  WindSpeed9am            7557 non-null   float64
12  WindSpeed3pm            7557 non-null   float64
13  Humidity9am             7557 non-null   float64
14  Humidity3pm             7557 non-null   float64
15  Pressure9am             7557 non-null   float64
16  Pressure3pm             7557 non-null   float64
17  Cloud9am                7557 non-null   float64
18  Cloud3pm                7557 non-null   float64
19  Temp9am                 7557 non-null   float64
20  Temp3pm                 7557 non-null   float64
21  RainYesterday           7557 non-null   str
22  RainToday               7557 non-null   str
dtypes: float64(16), str(7)
memory usage: 1.6 MB

```

We still have 7557 records, which should be enough to build a reasonably good model. You could always gather more data if needed by partitioning the data into similar locations or simply by updating it from the source to include a larger time frame.

2.7 Extracting a seasonality feature

Now consider the **Date** column. We expect the weather patterns to be seasonal, having different predictability levels in winter and summer for example.

There may be some variation with **Year** as well, but we'll leave that out for now. Let's engineer a **Season** feature from **Date** and drop **Date** afterward, since it is most likely less informative than season. An easy way to do this is to define a function that assigns seasons to given months, then use that function to transform the **Date** column.

2.7.1 Create a function to map dates to seasons

```
[71]: def date_to_season(date):
    month = date.month
    if (month == 12) or (month == 1) or (month == 2):
        return 'Summer'
    elif (month == 3) or (month == 4) or (month == 5):
        return 'Autumn'
    elif (month == 6) or (month == 7) or (month == 8):
        return 'Winter'
    elif (month == 9) or (month == 10) or (month == 11):
        return 'Spring'
```

2.8 Exercise 1: Map the dates to seasons and drop the Date column

Complete the code:

```
# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(...)

# Apply the function to the 'Date' column
df['Season'] = df['Date'].apply(date_to_season)

df=df.drop(columns=...)
df
```

```
[72]: # Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Apply the function to the 'Date' column
df['Season'] = df['Date'].apply(date_to_season)

df=df.drop(columns='Date')
df
```

```
[72]:
```

	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	\
64191	MelbourneAirport	11.2	19.9	0.0	5.6	8.8	
64192	MelbourneAirport	7.8	17.8	1.2	7.2	12.9	
64193	MelbourneAirport	6.3	21.1	0.0	6.2	10.5	
64194	MelbourneAirport	8.1	29.2	0.0	6.4	12.5	
64195	MelbourneAirport	9.7	29.0	0.0	7.4	12.3	
...	...	...	...	...	...	...	
80992	Watsonia	3.6	14.5	0.0	2.4	8.8	
80994	Watsonia	4.8	13.3	0.4	0.6	0.0	
80995	Watsonia	5.6	13.1	0.0	1.6	6.0	
80996	Watsonia	6.9	12.1	3.2	1.8	5.6	
80997	Watsonia	7.9	13.0	0.0	2.8	3.8	

	WindGustDir	WindGustSpeed	WindDir9am	WindDir3pm	...	Humidity3pm	\
64191	SW	69.0	W	SW	...	37.0	
64192	SSE	56.0	SW	SSE	...	43.0	
64193	SSE	31.0	E	S	...	35.0	
64194	SSE	35.0	NE	SSE	...	23.0	
64195	SE	33.0	SW	SSE	...	31.0	
...	...	...	...	...	...	...	
80992	NNE	41.0	ENE	NNE	...	66.0	
80994	NNW	24.0	NE	NNE	...	63.0	
80995	NNW	52.0	NE	N	...	67.0	
80996	SSW	24.0	WNW	SW	...	61.0	
80997	NNW	39.0	N	N	...	69.0	

	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	Temp9am	Temp3pm	\
64191	1005.1	1006.4	7.0	7.0	15.9	18.1	
64192	1018.0	1019.3	6.0	7.0	12.5	15.8	
64193	1020.8	1017.6	1.0	7.0	13.4	19.6	
64194	1016.2	1012.8	5.0	4.0	16.0	28.2	
64195	1011.9	1010.3	6.0	2.0	19.4	27.1	
...	...	...	...	...	...	...	
80992	1028.4	1025.0	1.0	7.0	5.2	13.8	
80994	1028.5	1025.1	7.0	7.0	5.6	12.4	
80995	1019.0	1014.0	1.0	7.0	8.8	11.6	
80996	1018.7	1017.3	2.0	7.0	7.9	11.0	
80997	1017.6	1015.3	7.0	7.0	9.0	11.7	

	RainYesterday	RainToday	Season
64191	No	Yes	Summer
64192	Yes	No	Summer
64193	No	No	Summer
64194	No	No	Summer
64195	No	No	Summer
...	...	...	...
80992	No	No	Winter
80994	No	No	Winter
80995	No	Yes	Winter
80996	Yes	No	Winter
80997	No	No	Winter

[7557 rows x 23 columns]

Looks like we have a good set of features to work with.

Let's go ahead and build our model.

But wait, let's take a look at how well balanced our target is.

2.9 Exercise 2. Define the feature and target dataframes

Complete the following code:

```
X = df.drop(columns='...', axis=1)
y = df['...']
```

```
[73]: df.columns
```

```
[73]: Index(['Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation', 'Sunshine',
        'WindGustDir', 'WindGustSpeed', 'WindDir9am', 'WindDir3pm',
        'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',
        'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm', 'Temp9am',
        'Temp3pm', 'RainYesterday', 'RainToday', 'Season'],
        dtype='str')
```

```
[74]: # Write your response.
features = ['Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation',
           'Sunshine',
           'WindGustDir', 'WindGustSpeed', 'WindDir9am', 'WindDir3pm',
           'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',
           'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm', 'Temp9am',
           'Temp3pm', 'RainYesterday', 'Season']
target = 'RainToday'

X = df[features]
y = df[target]
```

2.10 Exercise 3. How balanced are the classes?

Display the counts of each class.

Complete the following code:

```
... .value_counts()
```

```
[75]: # Write your response.
y.value_counts()
```

```
[75]: RainToday
No      5766
Yes     1791
Name: count, dtype: int64
```

2.11 Exercise 4. What can you conclude from these counts?

- How often does it rain annually in the Melbourne area?
- How accurate would you be if you just assumed it won't rain every day?
- Is this a balanced dataset?
- Next steps?

2.12 Write your response here and convert the cell to a markdown.

3 76% of the time is not raining in Melbourne area.

4 If we assume it will not rain today we will be 76% accurate since we have 76% of not raining

5 The dataset is not balanced (76% versus 24%)

6 We need to select a model that consider the dataset is not balanced and test the best model.

```
[76]: # Write your response.  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
↳stratify=y, random_state=42)
```

6.1 Define preprocessing transformers for numerical and categorical features

6.2 Exercise 6. Automatically detect numerical and categorical columns and assign them to separate numeric and categorical features

Complete the following code:

```
numeric_features = X_train.select_dtypes(include=['...']).columns.tolist()  
categorical_features = X_train.select_dtypes(include=['...', 'category']).columns.tolist()
```

```
[77]: # Write your response.  
numeric_features = X_train.select_dtypes(include=['number']).columns.tolist()  
categorical_features = X_train.select_dtypes(include=['object', 'category']).  
↳columns.tolist()
```

/tmp/ipykernel_325/3325895139.py:3: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
categorical_features = X_train.select_dtypes(include=['object',  
'category']).columns.tolist()
```

6.2.1 Define separate transformers for both feature types and combine them into a single preprocessing transformer

```
[78]: # Scale the numeric features  
numeric_transformer = Pipeline(steps=[('scaler', StandardScaler())])
```

```
# One-hot encode the categoricals
categorical_transformer = Pipeline(steps=[('onehot',
↳ OneHotEncoder(handle_unknown='ignore'))])
```

6.3 Exercise 7. Combine the transformers into a single preprocessing column transformer

Complete the following code:

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, ...),
        ('cat', categorical_transformer, ...)
    ]
)
```

```
[79]: # Write your response.
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

6.4 Exercise 8. Create a pipeline by combining the preprocessing with a Random Forest classifier

Complete the following code:

```
pipeline = Pipeline(steps=[
    ('preprocessor', ...),
    ('...', RandomForestClassifier(random_state=42))
])
```

```
[80]: # Write your response.
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42))
])
```

6.4.1 Define a parameter grid to use in a cross validation grid search model optimizer

```
[81]: param_grid = {
    'classifier__n_estimators': [50, 100],
    'classifier__max_depth': [None, 10, 20],
    'classifier__min_samples_split': [2, 5]
}
```

6.4.2 Pipeline usage in crossvalidation

Recall that the pipeline is repeatedly used within the crossvalidation by fitting on each internal training fold and predicting on its corresponding validation fold

6.5 Perform grid search cross-validation and fit the best model to the training data

6.5.1 Select a cross-validation method, ensuring target stratification during validation

```
[82]: cv = StratifiedKFold(n_splits=5, shuffle=True)
```

6.6 Exercise 9. Instantiate and fit GridSearchCV to the pipeline

Complete the following code:

```
grid_search = GridSearchCV(..., param_grid, cv=..., scoring='accuracy', verbose=2)
grid_search.fit(..., ...)
```

```
[83]: ### Write your response.
grid_search = GridSearchCV(estimator=pipeline, param_grid=param_grid, cv=cv,
    ↪scoring='accuracy', verbose=2)
grid_search.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=50; total time= 0.6s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=50; total time= 0.5s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=50; total time= 0.5s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=50; total time= 0.5s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=50; total time= 0.5s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=100; total time= 1.0s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=100; total time= 0.9s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=100; total time= 1.0s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=100; total time= 0.9s[CV] END
classifier__max_depth=None, classifier__min_samples_split=2,
classifier__n_estimators=100; total time= 0.9s[CV] END
classifier__max_depth=None, classifier__min_samples_split=5,
classifier__n_estimators=50; total time= 0.5s[CV] END
classifier__max_depth=None, classifier__min_samples_split=5,
classifier__n_estimators=50; total time= 0.5s[CV] END
classifier__max_depth=None, classifier__min_samples_split=5,
```


[illegible]


```
[83]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=None, shuffle=True),
                  estimator=Pipeline(steps=[('preprocessor',
                                             ColumnTransformer(transformers=[('num',
Pipeline(steps=[('scaler',
                  StandardScaler()))]),
['MinTemp',
'MaxTemp',
'Rainfall',
'Evaporation',
'Sunshine',
'WindGustSpeed',
'WindSpeed9am',
'WindSpeed3pm',
'Humidity9am',
'Humidity3pm',
'Pressure9am',
'Pr...

Pipeline(steps=[('onehot',
                  OneHotEncoder(handle_unknown='ignore'))]),
['Location',
'WindGustDir',
'WindDir9am',
'WindDir3pm',
'RainYesterday',
'Season']]])),
('classifier',
RandomForestClassifier(random_state=42))]),
param_grid={'classifier__max_depth': [None, 10, 20],
            'classifier__min_samples_split': [2, 5],
            'classifier__n_estimators': [50, 100]},
scoring='accuracy', verbose=2)
```

6.6.1 Print the best parameters and best crossvalidation score

```
[84]: print("\nBest parameters found: ", grid_search.best_params_)
print("Best cross-validation score: {:.2f}".format(grid_search.best_score_))
```

```
Best parameters found: {'classifier__max_depth': None,
'classifier__min_samples_split': 2, 'classifier__n_estimators': 100}
Best cross-validation score: 0.85
```

6.7 Exercise 10. Display your model's estimated score

Complete the following code:

```
test_score = grid_search.score(..., ...)
print("Test set score: {:.2f}".format(test_score))
```

```
[85]: ## Write your response.
test_score = grid_search.score(X_test, y_test)
print("Test set score: {:.2f}".format(test_score))
```

Test set score: 0.84

So we have a reasonably accurate classifier, which is expected to correctly predict about 84% of the time whether it will rain today in the Melbourne area.

But careful here. Let's take a deeper look at the results.

The best model is stored within the gridsearch object.

6.8 Exercise 11. Get the model predictions from the grid search estimator on the unseen data

Complete the following code:

```
y_pred = grid_search.predict(...)
```

```
[86]: ### Write your response.
y_pred = grid_search.predict(X_test)
```

6.9 Exercise 12. Print the classification report

Complete the following code:

```
print("\nClassification Report:")
print...(y_test, y_pred)
```

```
[87]: ## Write your response.
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
No	0.86	0.95	0.90	1154
Yes	0.76	0.50	0.60	358
accuracy			0.84	1512
macro avg	0.81	0.72	0.75	1512
weighted avg	0.83	0.84	0.83	1512

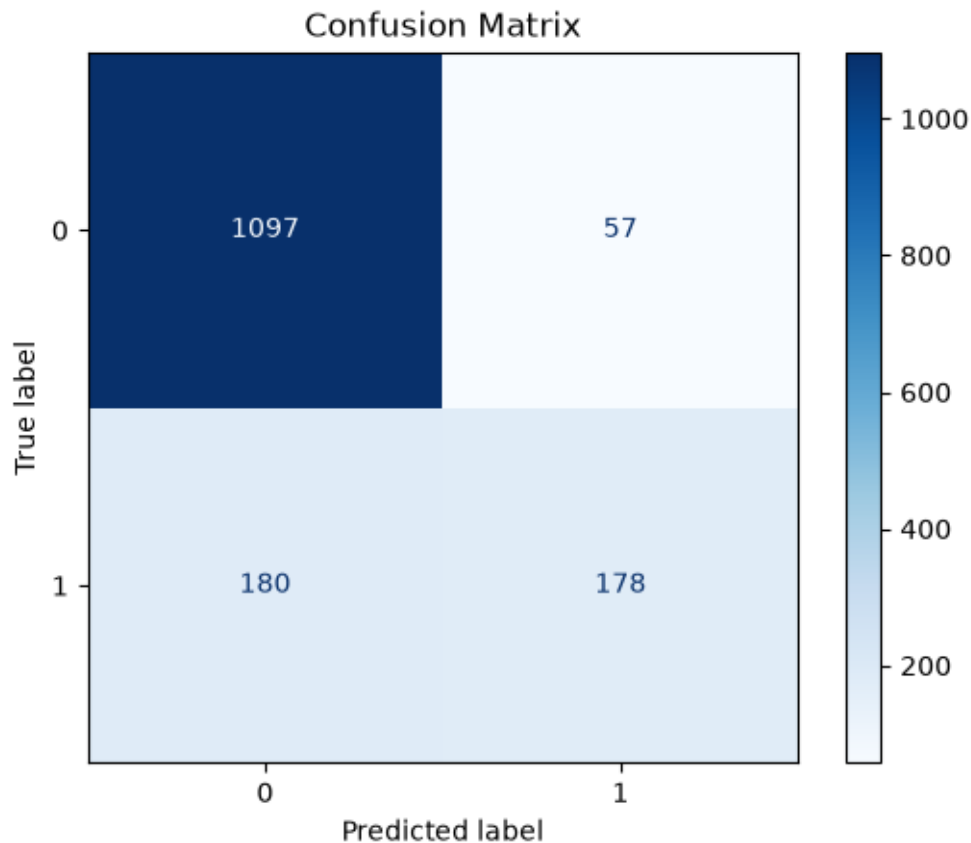
6.10 Exercise 13. Plot the confusion matrix

Complete the following code:

```
conf_matrix = ...(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=...)
disp.plot(cmap='Blues')
```

```
plt.title('Confusion Matrix')
plt.show()
```

```
[88]: ## Write your response.
conf_matrix = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix)
disp.plot(cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```



Let's consider whether the results indicate a good predictor of rainfall. *## Points to note - 2* What is the true positive rate? There will be a question on this in the assignment that follows.

[Click here for Hints](#)

Consider the confusion matrix or the classification report and calculate the true positive rate given the information.

6.11 Feature importances

Recall that to obtain the categorical feature importances, we have to work our way backward through the modelling pipeline to associate the feature importances with their original input vari-

ables, not the one-hot encoded ones. We don't need to do this for the numeric variables because we didn't modify their names in any way.

Remember we went from categorical features to one-hot encoded features, using the 'cat' column transformer.

Let's get all of the feature importances and associate them with their transformed features

6.12 Exercise 14. Extract the feature importances

Complete the following code:

```
feature_importances = grid_search.best_estimator_['classifier']. ...
```

```
[89]: ## Write your response.
feature_importances = grid_search.best_estimator_['classifier'].
    ↪feature_importances_
```

Now let's extract the feature importances and plot them as a bar graph.

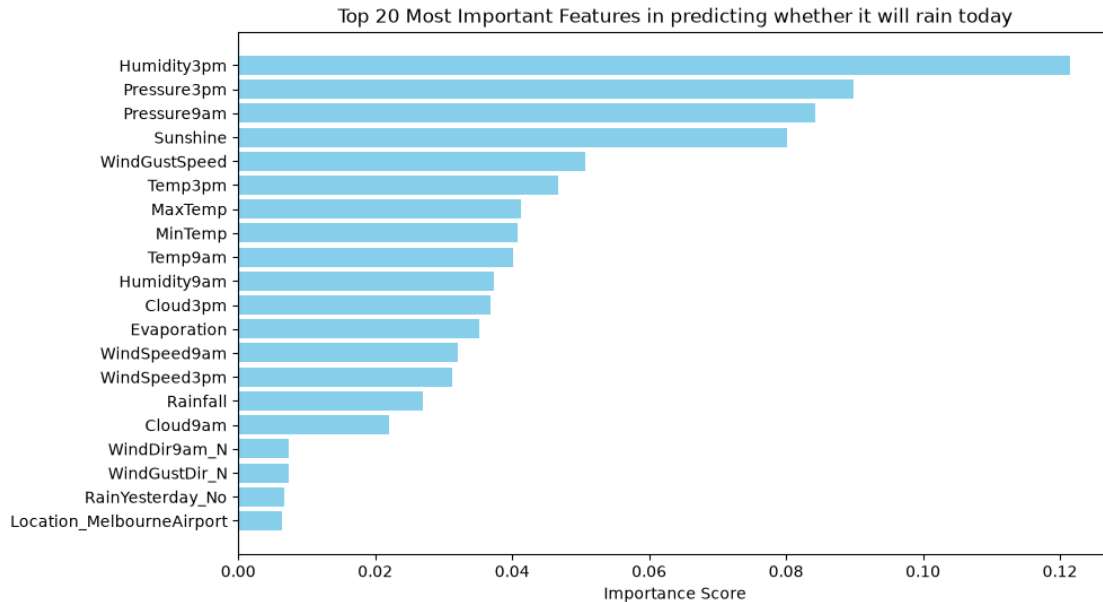
```
[90]: # Combine numeric and categorical feature names
feature_names = numeric_features + list(grid_search.
    ↪best_estimator_['preprocessor']
                                     .named_transformers_['cat']
                                     .named_steps['onehot']
                                     .
    ↪get_feature_names_out(categorical_features))

feature_importances = grid_search.best_estimator_['classifier'].
    ↪feature_importances_

importance_df = pd.DataFrame({'Feature': feature_names,
                             'Importance': feature_importances
                             }).sort_values(by='Importance', ascending=False)

N = 20 # Change this number to display more or fewer features
top_features = importance_df.head(N)

# Plotting
plt.figure(figsize=(10, 6))
plt.barh(top_features['Feature'], top_features['Importance'], color='skyblue')
plt.gca().invert_yaxis() # Invert y-axis to show the most important feature on_
    ↪top
plt.title(f'Top {N} Most Important Features in predicting whether it will rain_
    ↪today')
plt.xlabel('Importance Score')
plt.show()
```



6.13 Point to note - 3

Identify the most important feature for predicting whether it will rain based on the feature importance bar graph. There will be a question on this in the assignment that follows.

6.14 Try another model

Some thoughts. In practice you would want to try out different models and even revisit the data analysis to improve your model's performance. Maybe you can engineer better features, drop irrelevant or redundant ones, project your data onto a dimensional feature space, or impute missing values to be able to use more data. You can also try a larger set of parameters to define your search grid, or even engineer new features using cluster analysis. You can even include the clustering algorithm's hyperparameters in your search grid!

With Scikit-learn's powerful pipeline and GridSearchCV classes, this is easy to do in a few steps.

6.15 Exercise 15. Update the pipeline and the parameter grid

Let's update the pipeline and the parameter grid and train a Logistic Regression model and compare the performance of the two models. You'll need to replace the classifier with LogisticRegression. We have supplied the parameter grid for you.

Complete the following code:

```
# Replace RandomForestClassifier with LogisticRegression
pipeline.set_params(...=LogisticRegression(random_state=42))

# update the model's estimator to use the new pipeline
grid_search.estimator = ...
```

```
# Define a new grid with Logistic Regression parameters
```

```
param_grid = {  
    # 'classifier__n_estimators': [50, 100],  
    # 'classifier__max_depth': [None, 10, 20],  
    # 'classifier__min_samples_split': [2, 5],  
    'classifier__solver' : ['liblinear'],  
    'classifier__penalty': ['l1', 'l2'],  
    'classifier__class_weight' : [None, 'balanced']  
}
```

```
grid_search.param_grid = ...
```

```
# Fit the updated pipeline with LogisticRegression
```

```
model.fit(..., ...)
```

```
# Make predictions
```

```
y_pred = model.predict(X_test)
```

```
[91]: ## Write your response  
# Replace RandomForestClassifier with LogisticRegression  
pipeline.set_params(classifier=LogisticRegression(random_state=42))  
  
# update the model's estimator to use the new pipeline  
grid_search.estimator = pipeline  
  
# Define a new grid with Logistic Regression parameters  
param_grid = {  
    # 'classifier__n_estimators': [50, 100],  
    # 'classifier__max_depth': [None, 10, 20],  
    # 'classifier__min_samples_split': [2, 5],  
    'classifier__solver' : ['liblinear'],  
    'classifier__penalty': ['l1', 'l2'],  
    'classifier__class_weight' : [None, 'balanced']  
}  
  
grid_search.param_grid = param_grid  
  
# Fit the updated pipeline with LogisticRegression  
grid_search.fit(X_train, y_train)  
  
# Make predictions  
y_pred = grid_search.predict(X_test)
```

Fitting 5 folds for each of 4 candidates, totalling 20 fits

[CV] END classifier__class_weight=None, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.2s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:

```

FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
    warnings.warn(

[CV] END classifier__class_weight=None, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.1s
[CV] END classifier__class_weight=None, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.2s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
    warnings.warn(

```

```

[CV] END classifier__class_weight=None, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.2s
[CV] END classifier__class_weight=None, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.2s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(

[CV] END classifier__class_weight=None, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s
[CV] END classifier__class_weight=None, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s
[CV] END classifier__class_weight=None, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1

```



```

instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(

[CV] END classifier__class_weight=None, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s
[CV] END classifier__class_weight=None, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
    warnings.warn(

[CV] END classifier__class_weight=balanced, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.1s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
    warnings.warn(

[CV] END classifier__class_weight=balanced, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.2s
[CV] END classifier__class_weight=balanced, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.2s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use

```

```

'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
  warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
  warnings.warn(

[CV] END classifier__class_weight=balanced, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.2s
[CV] END classifier__class_weight=balanced, classifier__penalty=l1,
classifier__solver=liblinear; total time= 0.1s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1407:
UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is
deprecated. Please use l1_ratio only.
  warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
  warnings.warn(

[CV] END classifier__class_weight=balanced, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s
[CV] END classifier__class_weight=balanced, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s
[CV] END classifier__class_weight=balanced, classifier__penalty=l2,

```

```

classifier__solver=liblinear; total time= 0.1s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(

[CV] END classifier__class_weight=balanced, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s
[CV] END classifier__class_weight=balanced, classifier__penalty=l2,
classifier__solver=liblinear; total time= 0.1s

/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1381:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', l1_ratio set to a float between 0 and 1 instead of
penalty='elasticnet', and C=np.inf instead of penalty=None.
    warnings.warn(

```

6.15.1 Compare the results to your previous model.

Display the clasification report and the confusion matrix for the new model and compare your results with the previous model.

```
[92]: print(classification_report(y_test, y_pred))

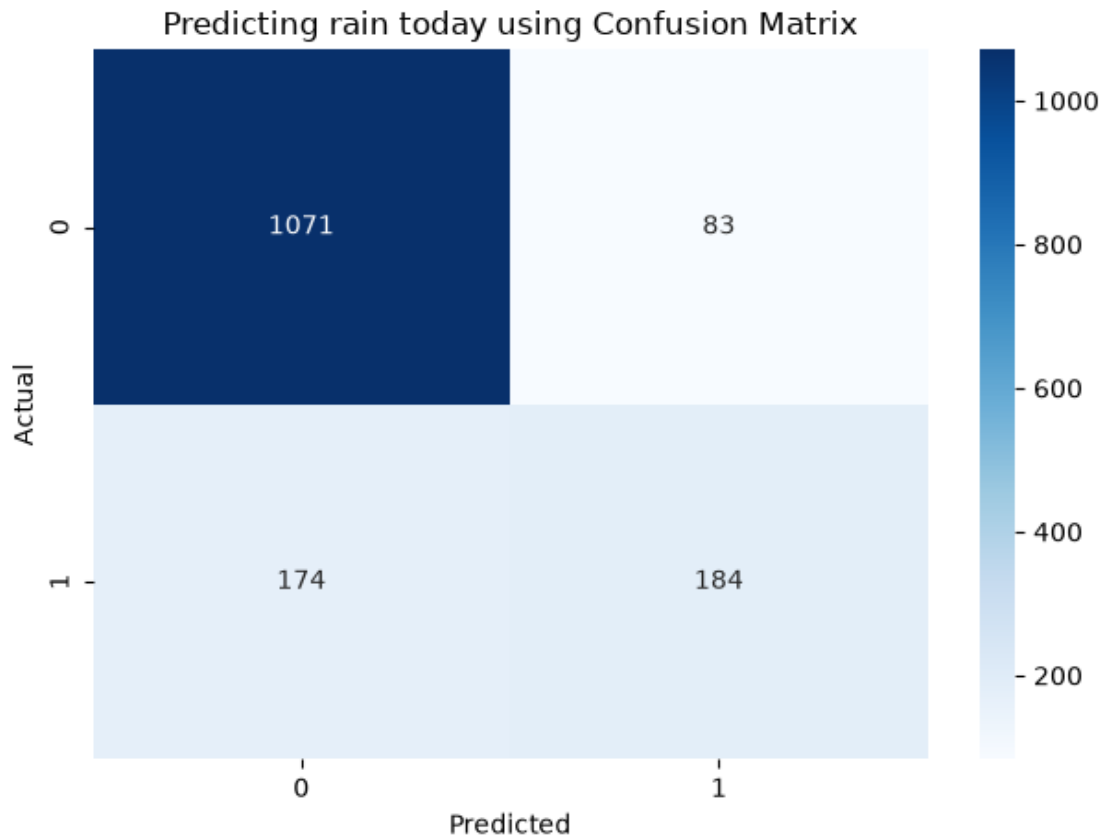
# Generate the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='d')

# Set the title and labels
plt.title('Predicting rain today using Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')

# Show the plot
plt.tight_layout()
plt.show()
```

	precision	recall	f1-score	support
No	0.86	0.93	0.89	1154
Yes	0.69	0.51	0.59	358
accuracy			0.83	1512
macro avg	0.77	0.72	0.74	1512
weighted avg	0.82	0.83	0.82	1512



What can you conclude about the model performances?

6.16 Points to note - 4

Compare the accuracy and true positive rate of rainfall predictions between the LogisticRegression model and the RandomForestClassifier model.

Note: Make sure to provide the answer in the form of a list using either bullets or numbers.

There will be a question on this in the assignment that follows.

[Click here for Hints](#)

Compare the accuracy percentages of both the classifiers.

Provide the details of the number of correct predictions.

Provide the true positive rate of LogisticRegression Classifier.

6.16.1 Congratulations! You've made it the end of your final project!

Well done! You now have some great tools to use for tackling complex real-world problems with machine learning.

6.17 Author

Jeff Grossman

6.17.1 Other Contributor(s)

Abhishek Gagneja

© IBM Corporation. All rights reserved.